The Islamia University of Bahawalpur
Department of Information Technology

INFT-4041 · BSIT · 5th Semester · Fall 2026

E-Commerce Application Development

Chapter

19

Extending the Platform: Your First Module

Part V — Running and Extending the Shop

Taught in	Week 14
Laboratory	A shipping method and a dashboard widget, written from nothing
Assessment	—

This is a class extract. It is *Chapter 19* of the unified handout, printed on its own so that a week's reading is a week's reading. Page numbers restart at 1 and are local to this extract; chapter, figure and table numbers are the ones the full volume gives them, so a reference you follow here lands in the same place there.

Cross-references to other chapters print as their numbers in the full volume. For the whole course in one file — front matter, the HEC tracing table, all twenty-three chapters and all seven appendices — see `ECAD_Fall2026_Handout.pdf`.

Dr. Mustafa Hameed · Department of Information Technology

Generated from `parts/` by `makechapters.py`; do not edit this PDF's source by hand

Chapter 19

Extending the Platform: Your First Module

Learning Outcomes

By the end of this chapter you will be able to:

- **Explain** what an extension is made of, and **state** why each part exists.
- **Write** a storefront module and a shipping method that add behaviour without editing anything the platform ships.
- **Install** an extension the way the marketplace installer does, in three steps you can read.
- **Measure** what a module costs a page, and **account** for every query it adds.
- **Judge** when to extend, when to configure, and when to refuse.

Before You Start

Chapter 8: themes, templates and overrides. Chapter 11: the shipping quote interface and tax classes. Chapter 6: counting the queries a page runs. Appendix B for enough PHP and Twig to read the files.

Key Terms in This Chapter

extension • module • extension point • namespace • autoloading • manifest • controller • model • language file • module instance • layout position • upgrade safety • permission check

19.1 Why Not Just Edit the File

Every feature in this chapter could be added faster by opening the shop's own code and changing it. The reason not to is one sentence long, and it arrives about eighteen months later: **the next upgrade overwrites your file, and nothing tells you which ones you changed.**

The three ways to change what a shop does, in order of preference

1. **Configure it.** If a setting exists, use it. Chapter 11's tax rules and Chapter 17's widgets are settings, and a setting survives everything.
2. **Extend it.** Add a directory under `extension/` that the platform loads. Nothing it ships is touched, the upgrade leaves your directory alone, and `git status` tells you exactly what you added.
3. **Edit it.** Only when the first two cannot reach, and then with the change recorded somewhere an upgrade will make you re-read.

Most commissioned work on this platform is the second. That is what the department's title adds to HEC's bare "E-Commerce", and it is the difference between configuring a shop and developing one.

19.2 What an Extension Is Made Of

Extension, Module, Instance

An **extension** is a directory under `extension/` whose internal shape mirrors the shop's own — an `admin/` half, a `catalog/` half, and a manifest. It may provide several things at once: this chapter's provides a module and a shipping method.

A **module** is a block of storefront output that a layout can place. It has two controllers: one that renders it for a customer, one that lets the shopkeeper configure it.

An **instance** is one placement of a module, with its own settings. The same module may appear three times on a page with three different configurations, which is why its render method is handed its settings rather than reading them from the global configuration.

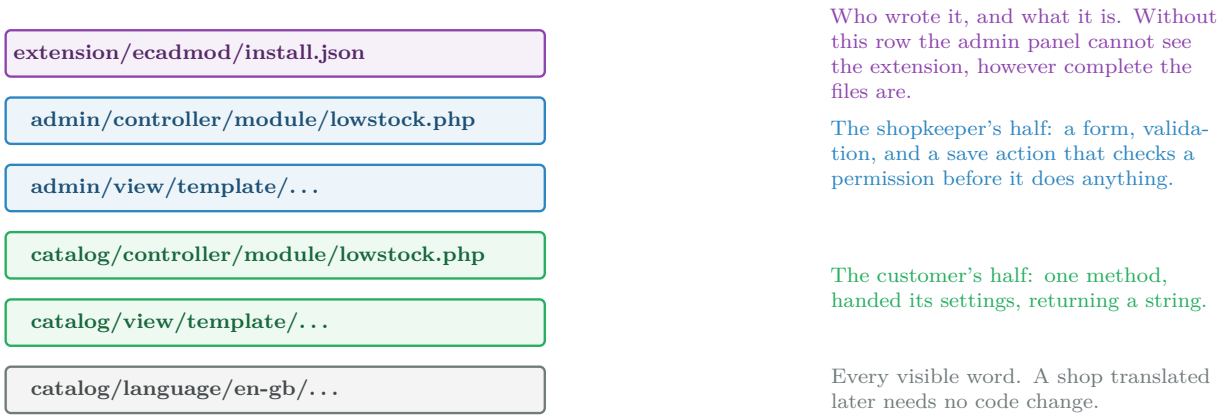


Figure 19.1: The twelve files of this chapter's extension. The namespace of every class is its path, which is how the loader finds it and the first thing to get wrong.

The namespace is the path, and the error message is unhelpful

The class

`Opencart\Catalog\Controller\Extension\Ecadmod\Module`

lives in `extension/ecadmod/catalog/controller/module/`. Get a single segment wrong — the wrong case, a missing `Extension`, the folder named differently from the namespace — and the loader reports that the class does not exist.

Which is true, and tells you nothing about why. **When an extension does not load, check the namespace against the path before anything else**; it is the cause more often than every other possibility combined.

19.3 The Storefront Half, in Full

The whole customer-facing side of the module is one method. It is worth reading in full, because almost nothing in it is specific to this module.

```
namespace Opencart\Catalog\Controller\Extension\Ecadmod\Module;

class LowStock extends \Opencart\System\Engine\Controller {
    public function index(array $setting): string {
        $this->load->language('extension/ecadmod/module/lowstock');

        $data['heading_title'] = $this->language->get('heading_title');
        $threshold = (int)($setting['threshold'] ?? 5);
```

```
        $limit     = (int)($setting['limit'] ?? 4);

        $query = $this->db->query("SELECT p.product_id,
            p.quantity, pd.name FROM ...
            WHERE p.status = '1' AND p.quantity > '0'
              AND p.quantity <= '" . (int)$threshold . "'
            ORDER BY p.quantity ASC LIMIT " . (int)$limit);
```

and then, in the same method, the rows become view data and the view becomes a string:

```
        $data['products'] = [];

        foreach ($query->rows as $row) {
            $data['products'][] = [
                'name'     => $row['name'],
                'quantity' => $row['quantity'],
                'href'     => $this->url->link('product/product',
                    'language=' . $this->config->get('config_language')
                    . '&product_id=' . $row['product_id'])
            ];
        }

        if (!$data['products']) { return ''; }

        return $this->load->view(
            'extension/ecadmod/module/lowstock', $data);
    }
}
```

Four decisions in thirty lines

1. **It takes `$setting` as an argument** rather than reading `$this->config`. That is what allows the module to be placed twice with different thresholds.
2. **It returns a string and never prints.** The layout decides where the string goes; a module that echoes has taken a decision that is not its own.
3. **It returns `''` when there is nothing to say.** An empty panel with a heading is worse than no panel.
4. **Every integer is cast.** `(int)$threshold` inside a query string is the difference between a parameter and an injection — and Chapter 16 showed that the platform is careful about this everywhere, which means your extension is now the weakest part.

19.4 Installing It, in Three Steps You Can Read

The marketplace installer is a button. What it does is three things, and doing them by hand once is the fastest way to understand why an extension that looks installed sometimes is not.

What `code/m19_modules.py -install` performs

1. **Copy the files** to `extension/ecadmod/` and make them readable by the web server. Twelve files.
2. **Insert the manifest row** into `oc_extension_install` — name, author, version, link. **Without this the admin panel cannot see the extension at all**, however complete the files are. This is the step people miss, and the symptom is an extension that is plainly on disk and absent from every list.
3. **Insert one row per thing provided** into `oc_extension`: `(ecadmod, module, lowstock)` and `(ecadmod, shipping, zonerate)`. This is what makes each appear in its own list rather than all of them in one.

Uninstalling is the same three in reverse, and the script does that too — because an extension you cannot remove cleanly is one you cannot test.

19.5 What a Module Costs

The objection to building features as modules is that they must be slow. It is worth measuring rather than assuming, in both directions.

Measured: The same home page with 0, 1, 3 and 5 instances

From `code/m19_modules.py -measure` against the stack in Appendix A. Each instance is a separate placement of the same module on the home layout.

Instances	Page	Queries	ms each	queries each
0	76 ms	117	—	—
1	82 ms	121	5.8	**4.0**
3	90 ms	129	4.7	**4.0**
5	96 ms	137	4.0	**4.0**

The cost is linear and small: about 4 ms and exactly four queries per instance. Five instances added 20 ms to a 76 ms page. A module is not expensive.

But the module contains *one* query. The other three are the framework's, and reading them from the slow log names each one:

`SELECT * FROM oc_module WHERE module_id=16`	this instance's settings
`SELECT * FROM oc_translation WHERE route=...`	its translations
`SELECT p.product_id, p.quantity ...`	**the one I wrote**
`SELECT * FROM oc_theme WHERE route=...`	its template override

Three quarters of what a module costs is the extension mechanism itself, spent finding your settings, your words and your template. That is the price of the arrangement that lets you place the module twice, translate it without touching code, and override its template from a theme.

It is also the explanation for Chapter 6's 144 queries on a category page, which was never explained at the time: a page is mostly modules, and every one of them pays this toll.

When to extend, and when to refuse

Extend when the behaviour is yours, the platform offers a point for it, and you can state what the shop would do without it.

Configure instead when a setting already reaches. Rewriting a setting as a module is the commonest waste in commissioned work.

Refuse when the request is for something the platform deliberately does not do, and the reason it does not is a good one. "Store the card number so customers do not have to retype it" is a module somebody will ask you to write, and Chapter 14 is the reason to say no — in writing, with the consequence named.

Common Pitfalls

- **Editing a shipped file** because it is faster today.
- **A namespace that does not match the path.** The error says the class does not exist, which is unhelpfully true.
- **Forgetting the `oc_extension_install` row**, which leaves an extension on disk and invisible.
- **Reading `$this->config` in a module's `render` method** instead of its `$setting` argument, so it cannot be placed twice.
- **Omitting the permission check in a save action.** The form is only shown to an administrator; the action is a URL.
- **Interpolating an uncast value into SQL.** The platform is careful everywhere else, so your extension becomes the weakest part.
- **Returning a tax-inclusive cost from a shipping quote**, which the totals pipeline then taxes again (Chapter 11).
- **Hard-coding visible text** in a controller or template rather than a language file.

Try It Yourself: Write, install, measure, and then break it

```
# --- 1. Read the twelve files before running anything ---------------
cd code
find modules/ecadmod -type f | sort
# Match each class's namespace to its path. Say the rule out loud.

# --- 2. Install it the way the marketplace would --------------------
python m19_modules.py --install

# --- 3. Place it, and look ------------------------------------------
python m19_modules.py --place 1
# Open the storefront. Nothing shows -- the demo products all have
# high stock. Fix that by raising the threshold, and say why an
# empty module returning '' is better than an empty panel.

# --- 4. Predict the cost, then measure it ---------------------------
# How many queries does ONE instance add? Commit to a number.
python m19_modules.py --measure

# --- 5. Account for all four ----------------------------------------
docker compose exec db mariadb -uroot -prootpw -e \
  "SELECT LEFT(CONVERT(sql_text USING utf8),80) FROM mysql.slow_log;"
# One is yours. Name the other three and say what each is for.

# --- 6. Break the namespace on purpose ------------------------------
# Rename the class to Lowstock (small s) and reload. Read the error.
# This is the error you will see most often; meet it deliberately now.

# --- 7. Configure your shipping method ------------------------------
# Admin: Extensions > Extensions > Shipping > Zone Rate > Edit.
# Rates one per line as zone_id:cost. Place an order and check the
# quote, then check oc_order_total for how the tax was applied.

# --- 8. Add the free-delivery threshold and test both sides ---------
# One order below it, one above. Verify from oc_order_total, not
# from the screen.

# --- 9. Remove it completely ----------------------------------------
python m19_modules.py --remove
# Then confirm the shop is exactly as it was: no files, no rows,
```

```
# no settings. An extension you cannot remove is one you cannot test.
```

What to notice. Step 5 is the chapter. Your module is one query; it costs four. Knowing which three are the mechanism's is what lets you judge whether a page with twenty modules is badly written or merely busy.

Checkpoint — Test Yourself

1. Five instances of a module added 20 ms and 20 queries to a 76 ms page. A colleague concludes modules are expensive and proposes editing the theme instead. Give your answer, with the per-instance breakdown, and say what you would measure before agreeing to anything.
2. A module's render method reads its threshold from `$this->config->get('module_lowstock_threshold')` instead of from its `$setting` argument. The shop works. Describe exactly when it stops working and what the shopkeeper will report.
3. You are asked to add a field to the checkout that the platform does not offer. Set out how you would decide between configuring, extending and editing, what you would need to know first, and what you would write down whichever you chose.

Chapter Summary

- Configure first, extend second, edit last. An extension is a directory the upgrade leaves alone; an edit is a file the upgrade overwrites without telling you.
- An extension mirrors the shop's shape: an admin half, a catalog half, a manifest, and language files for every visible word.
- The namespace of every class is its path. A mismatch reports that the class does not exist, and is the commonest cause by a wide margin.
- A module's render method takes its settings as an argument, returns a string, returns `''` when it has nothing to say, and casts every value it puts in a query.
- Installing is three steps: the files, a manifest row in `oc_extension_install`, and one row per provided thing in `oc_extension`. Miss the second and the extension is on disk and invisible.
- Measured: 4 ms and exactly 4 queries per module instance; five instances added 20 ms to a 76 ms page. Modules are cheap.
- Measured: of those four queries, *one* is the module's. The other three find its settings, its translations and its template — the price of being placeable, translatable and overridable, and the reason a category page runs 144 queries.
- A shipping method's whole interface is one method that is handed an address and returns a quote without tax, or nothing at all.
- Refusing is a professional act. "Store the card number" is a module somebody will ask for; decline it in writing, with the consequence named.

Review Questions

1. (MCQ) One instance of the measured module added how many database queries? (a) 1; (b) 2; (c) 4; (d) it varies with the catalogue.

2. (MCQ) An extension whose files are all present but which appears in no admin list is most likely missing: (a) a language file; (b) its `oc_extension_install` row; (c) file permissions; (d) a template.

3. (Short) Explain why a module's render method is handed its settings instead of reading the global configuration.

4. **(Short)** Give the rule relating a class's namespace to its file path, and the symptom of breaking it.

5. **(Short)** Why must a shipping quote return a cost without tax? Refer to Chapter 11's totals pipeline.

6. **(Applied)** You are asked for a module showing "customers who bought this also bought". Specify it: what it queries, where its settings live, how many queries it will add per page, how you would keep that number from growing with the catalogue, and what you would measure before and after.

7. **(Applied)** A client wants a field on the product page that their staff fill in and customers see. Compare doing this with a product attribute (configuration), a module (extension) and a template edit. Give the work, the upgrade risk and the maintenance burden of each, and recommend one with your reasons.